



Stack and Queue

Stacks and Queues

- A stack is a last in, first out (LIFO) data structure
 - Items are removed from a stack in the reverse order from the way they were inserted
 - A queue is a first in, first out (FIFO) data structure □ Items are removed from a queue in the same order as they were inserted



STACK

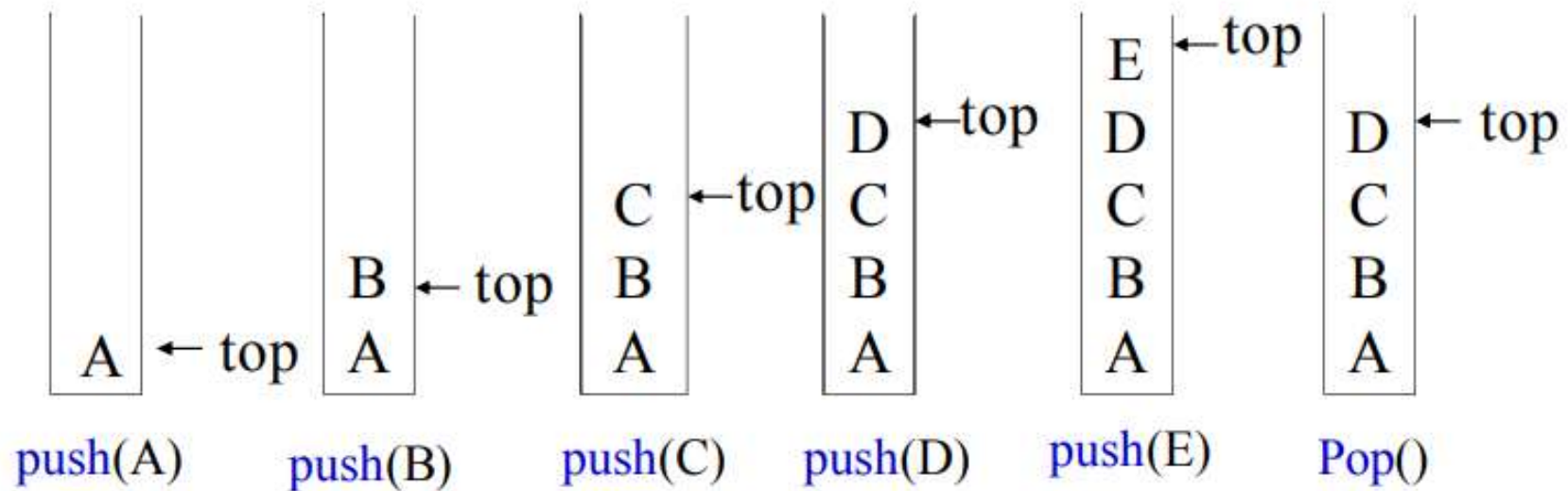
Stack

- A stack is a list with the restriction that insertions and deletions can be performed in only one position, namely, the top of the stack.
- The operations: push (insert) and pop (delete)

push(A) push(B) push(C) push(D) push(E) Pop()

Stack

- A stack is a list with the restriction that insertions and deletions can be performed in only one position, namely, the top of the stack.
- The operations: push (insert) and pop (delete)



Stack

- Show the status of a STACK implemented by an array of size, $m=5$ for the operations:
 - `push(10), push(20), pop(), push(30), push(40), pop(), pop(), pop()`.

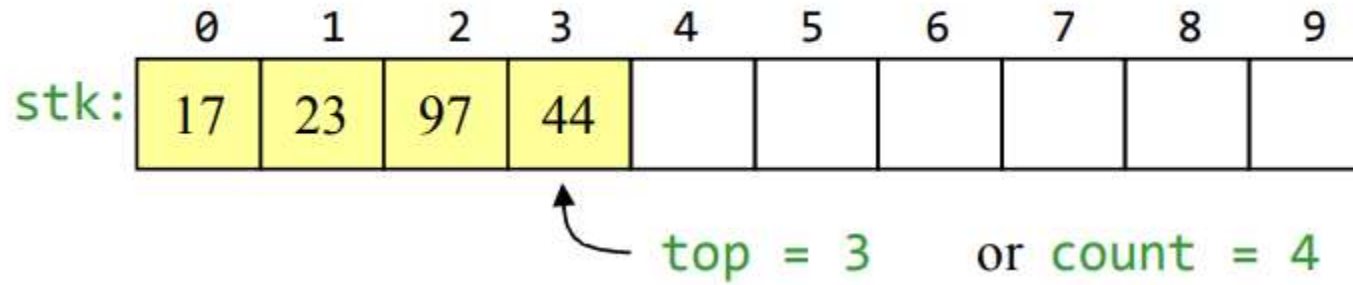
Applications of Stack

- Direct applications
 - Page-visited history in a Web browser
 - Undo sequence in a text editor
 - Saving local variables when one function calls another, and this one calls another, and so on. ☐
- Indirect applications
 - Auxiliary data structure for algorithms
 - Component of other data structures

Array Implementation of Stacks

- To implement a stack, items are inserted and removed at the same end (called the top)
- To use an array to implement a stack, you need both the array itself and an integer
 - The integer tells you either:
 - ◆ Which location is currently the top of the stack, or
 - ◆ How many elements are in the stack

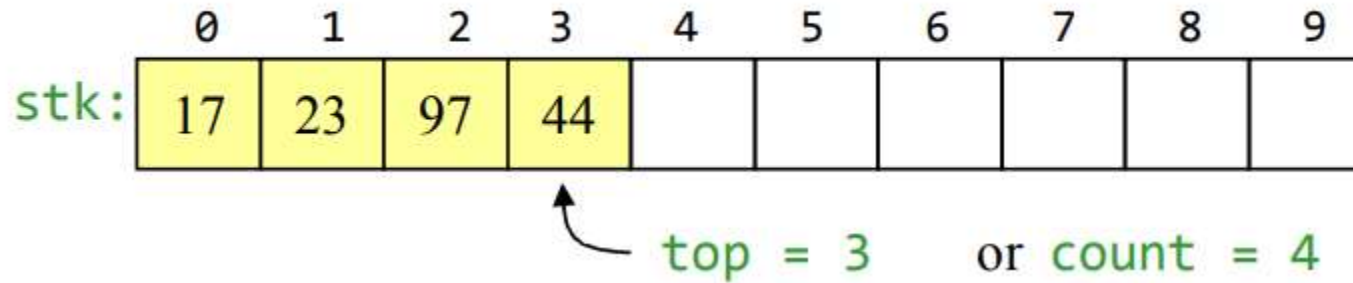
Array Implementation of Stacks



The functions after being called, will look like this:

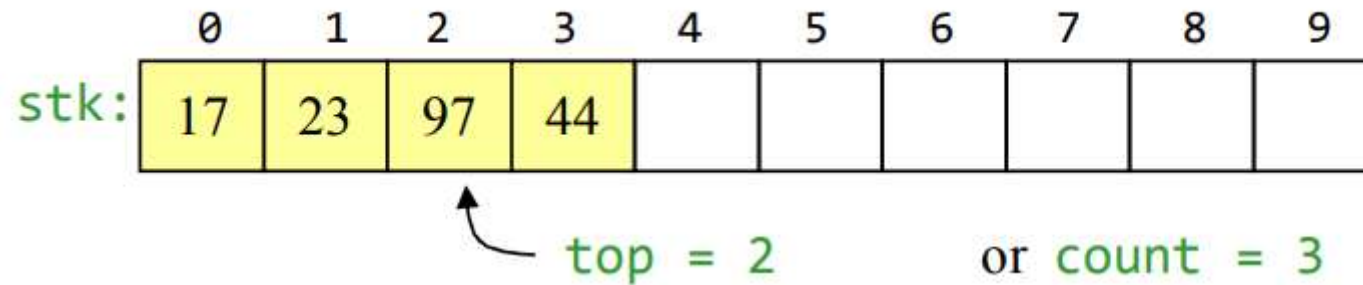
- `Push(44);`
- `X=Pop();`

Stacks by Array: Push and Pop



- If the bottom of the stack is at location 0, then an empty stack is represented by **top = -1** or **count = 0**
- To **add (push)** an element, either:
 - Increment **top** and store the element in **stk[top]**, or
 - Store the element in **stk[count]** and increment **count**
- To **remove (pop)** an element, either:
 - Get the element from **stk[top]** and decrement **top**, or
 - Decrement **count** and get the element in **stk[count]**

Stacks by Array: After Popping

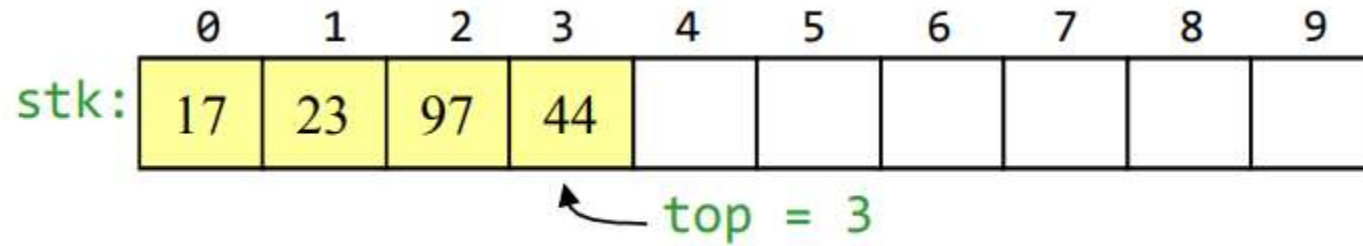


- When you pop an element, do you just leave the “deleted” element sitting in the array?
- The surprising answer is, “**it depends**”
 - If this is an array of **primitives**, then doing anything more is just a waste of time
 - If the array contains **objects**, you should set the delete array element to null

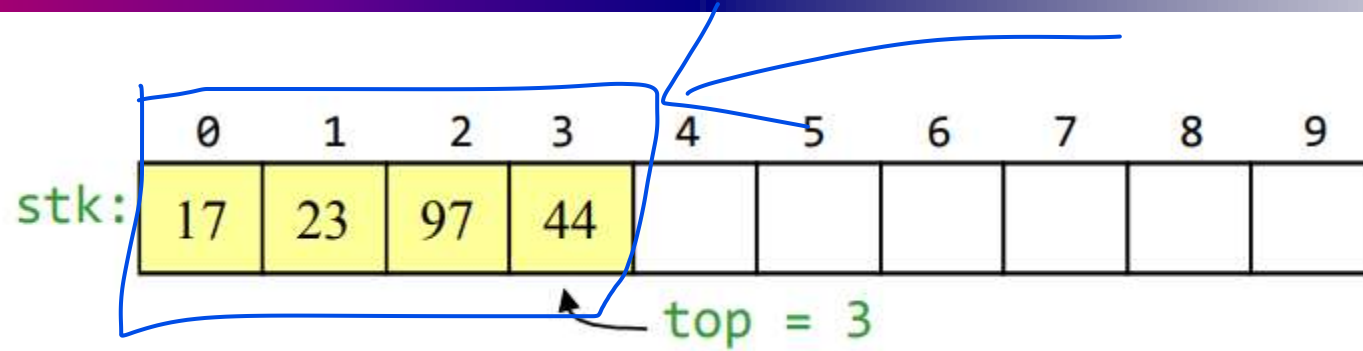
Stacks by Array: Error Checking

- There are two stack errors that can occur:
 - **Underflow**: trying to pop (or peek at) an empty stack
 - **Overflow**: trying to push onto an already full stack
- **For underflow**, you should check whether the stack is empty or not
 - Use if condition
- **For overflow**, you could do the same thing
 - Or, you could copy everything into a new, larger array

Let us write the Code for Stack



Let us write the Code for Stack



```
void push(int x){
```

```
    if(top >= n-1)
```

```
        printf("\n STACK is over flow");
```

```
    else {
```

```
        top++;
```

```
        stk[top] = x;
```

```
    }
```

```
}
```

```
int pop() {
```

```
    int y;
```

```
    if(top <= -1)
```

```
        printf("\n Stack is under flow");
```

```
    else {
```

```
        y = stk[top];
```

```
        top--;
```

```
        return y;
```

```
    }
```

```
}
```

Linked List Implementation of Stacks

- With a linked-list representation, overflow will not happen (unless you exhaust memory, which is another kind of problem)
- Underflow can happen, and should be handled the same way as for an array implementation

Let us write the code for SLL : Push

Let us write the code for SLL : Push

```
struct Node {  
    int value;  
    struct Node* next;  
};  
struct Node* top;
```

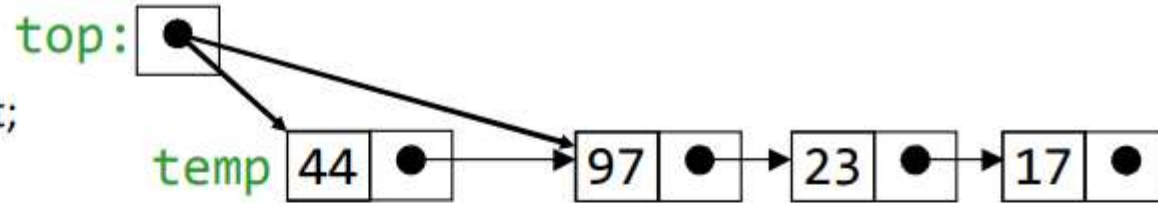
```
void push(int data) {  
    struct Node* temp;  
    temp = (struct Node *)malloc(sizeof(struct Node));
```

```
    // Check if memory(heap) is full.
```

```
    if (!temp){  
        cout << "\n Heap Overflow";  
        exit(1);  
    }
```

```
    temp->value = data;  
    temp->next = top;  
    top = temp;
```

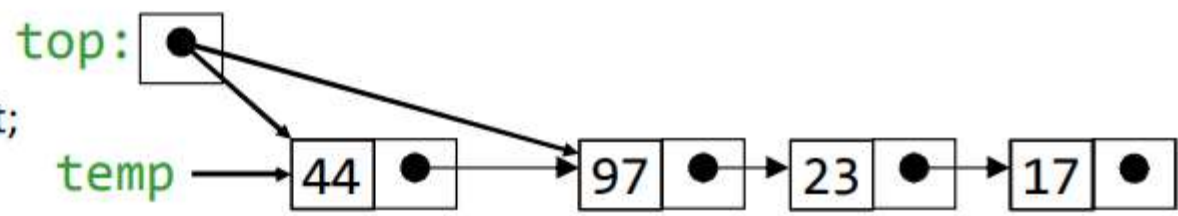
```
}
```



Let us write the code for SLL : Pop

Let us write the code for SLL : Pop

```
struct Node {
    int value;
    struct Node* next;
};
struct Node* top;
int pop(){
    struct Node* temp;
    int data;
    if (top == NULL) {
        cout << "\n Stack Underflow" << endl;
        exit(1); }
    else {
        data = top->value;
        temp = top;
        top = top->next;
        free(temp);
        return data;
    }
}
```



The diagram illustrates a Singly Linked List (SLL) with four nodes. Each node is represented as a box divided into two parts: a value field and a next pointer field. The values in the nodes are 44, 97, 23, and 17, respectively. A pointer labeled 'top' points to the first node (44). A pointer labeled 'temp' also points to the first node (44). The 'next' pointer of the first node points to the second node (97), the second node points to the third node (23), and the third node points to the fourth node (17). The fourth node's 'next' pointer is not explicitly shown, but it would typically point to NULL.



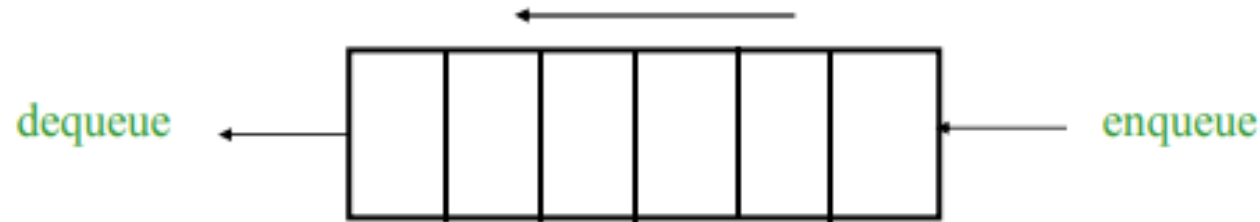
QUEUE

Queue: First In First Out

- A Queue is an ordered collection of items from which items may be removed at one end (called the front of the queue) and into which items may be inserted at the other end (the rear of the queue).
- The operations: **enqueue** (insert) and **dequeue** (delete)

Queue: First In First Out

- A Queue is an ordered collection of items from which items may be removed at one end (called the front of the queue) and into which items may be inserted at the other end (the rear of the queue).
- The operations: **enqueue** (insert) and **dequeue** (delete)



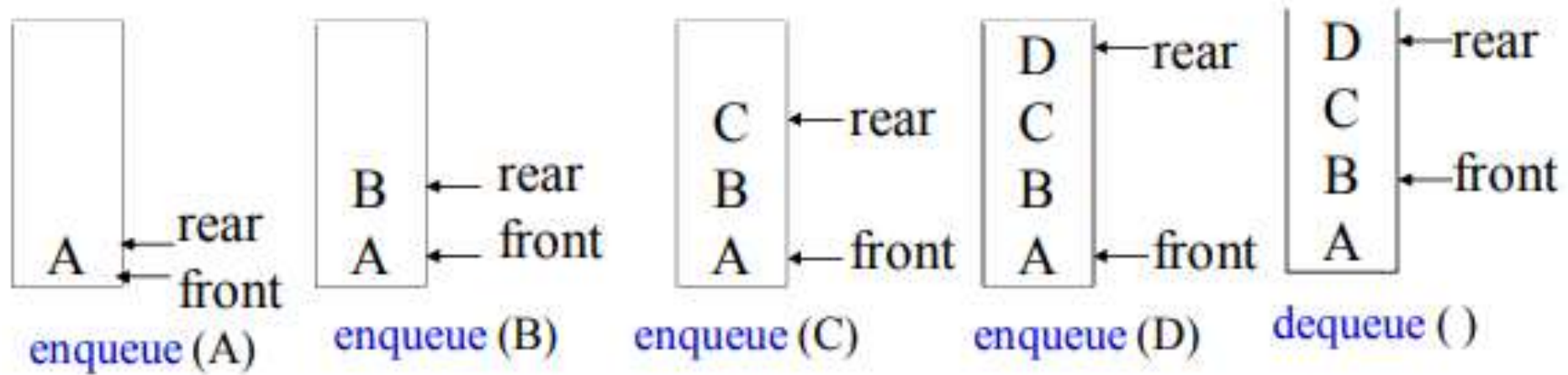
Queue: First In First Out

- A Queue is an ordered collection of items from which items may be removed at one end (called the front of the queue) and into which items may be inserted at the other end (the rear of the queue).
- The operations: **enqueue** (insert) and **dequeue** (delete)

enqueue (A) enqueue (B) enqueue (C) enqueue (D) dequeue ()

Queue: First In First Out

- A Queue is an ordered collection of items from which items may be removed at one end (called the front of the queue) and into which items may be inserted at the other end (the rear of the queue).
- The operations: **enqueue** (insert) and **dequeue** (delete)



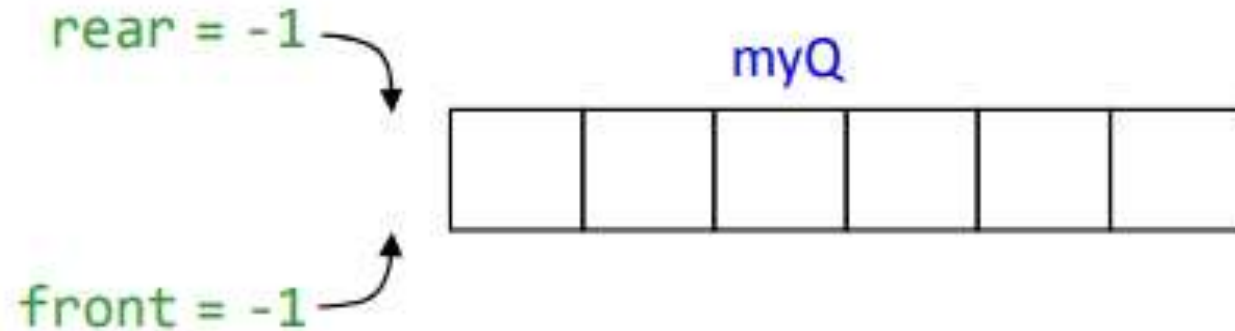
Queue: First In First Out

Application of Queues

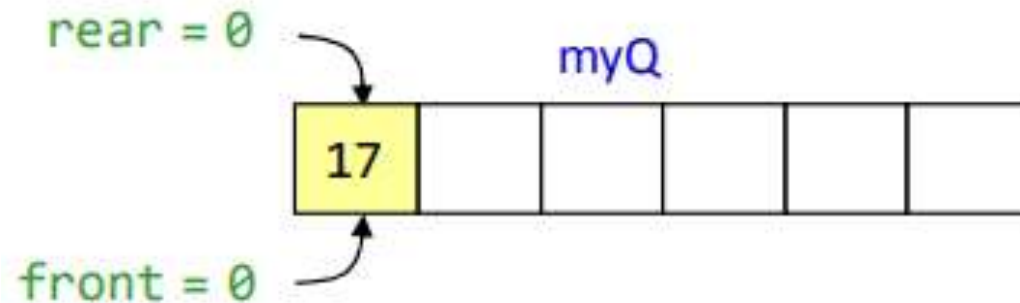
- Direct applications
 - Waiting lists, bureaucracy
 - Access to shared resources (e.g., printer)
 - Multiprogramming
- Indirect applications
 - Auxiliary data structure for algorithms
 - Component of other data structures

Array Implementation of Queue

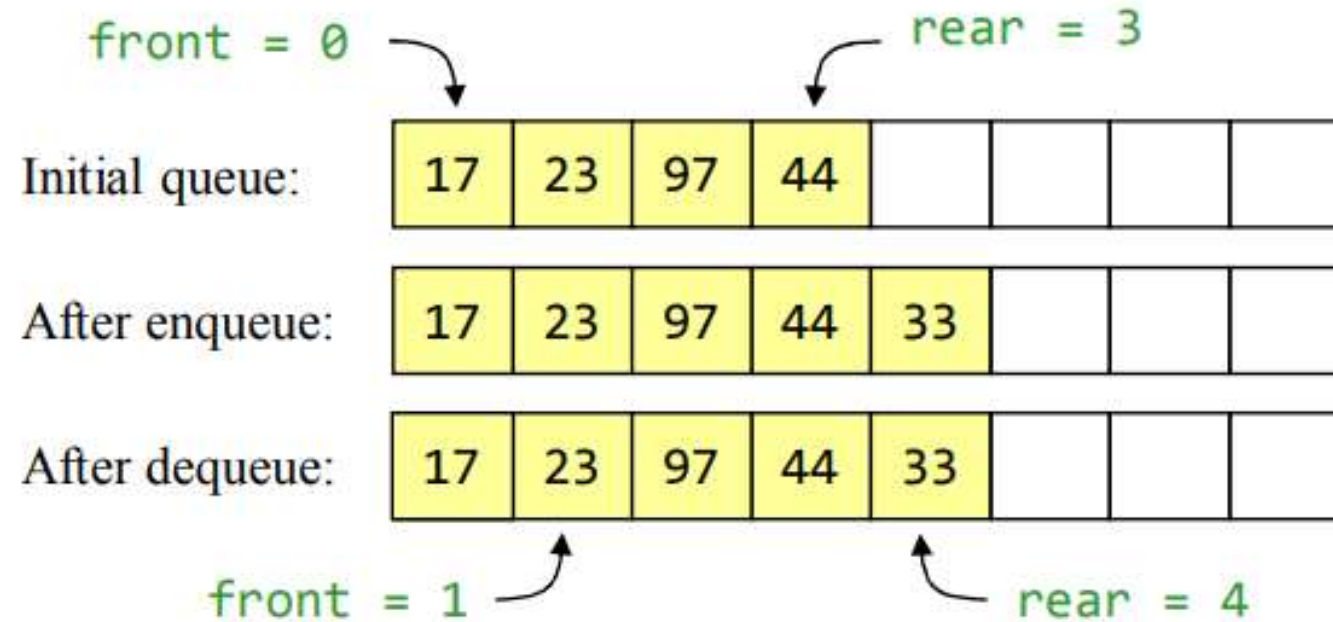
- Initial Queue, that is Empty Queue



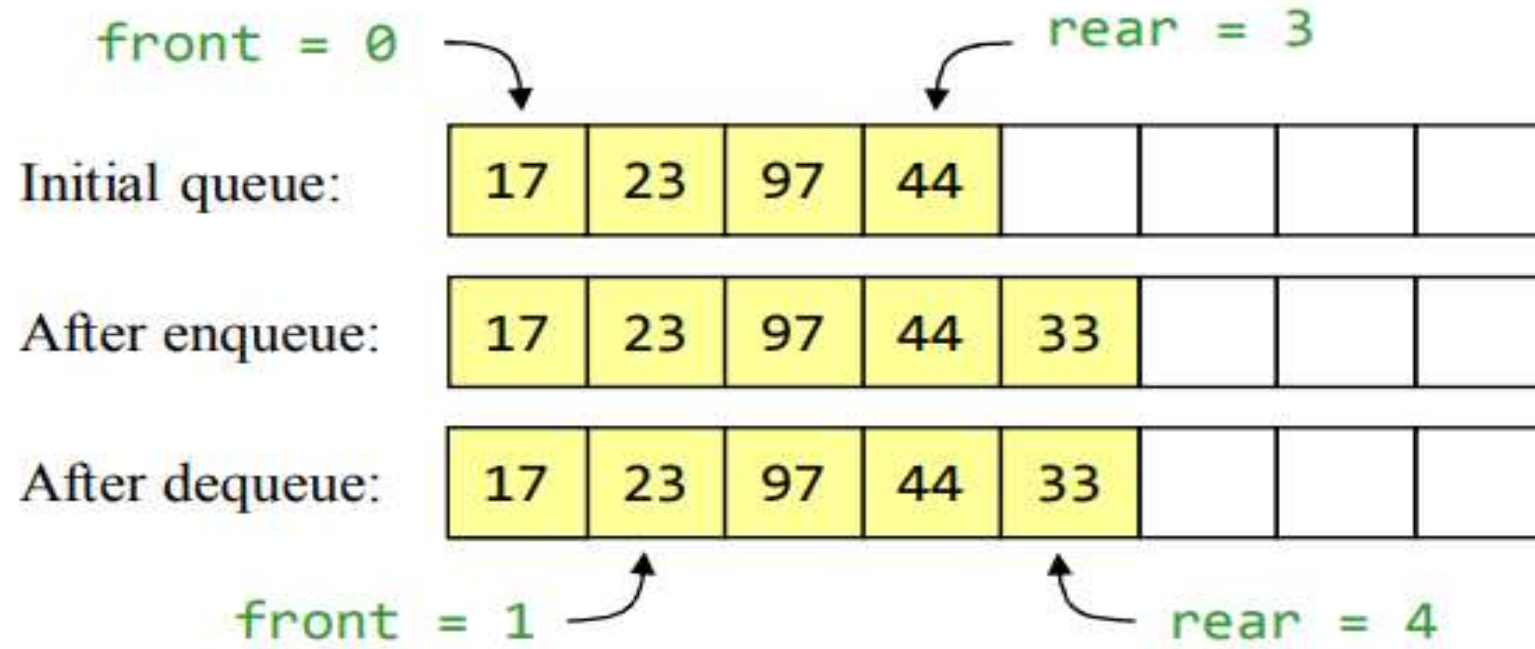
- After inserting 1st element in an Empty Queue, Set **front** = **rear** = 0



Array Implementation of Queues



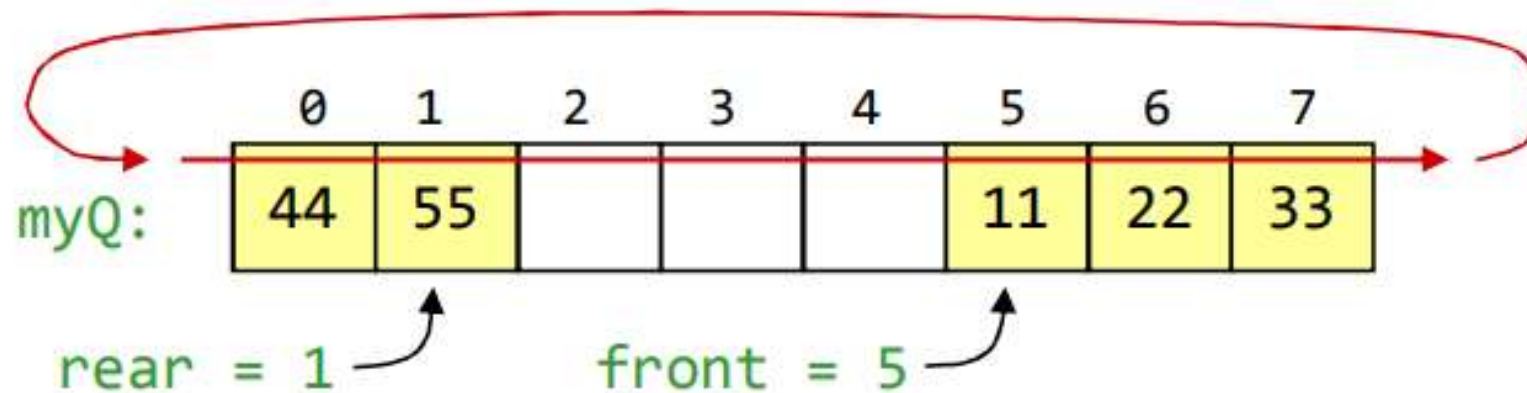
Array Implementation of Queues



- Notice how the array contents “crawl” to the right as elements are enqueued and dequeued
- This will be a problem after a while!

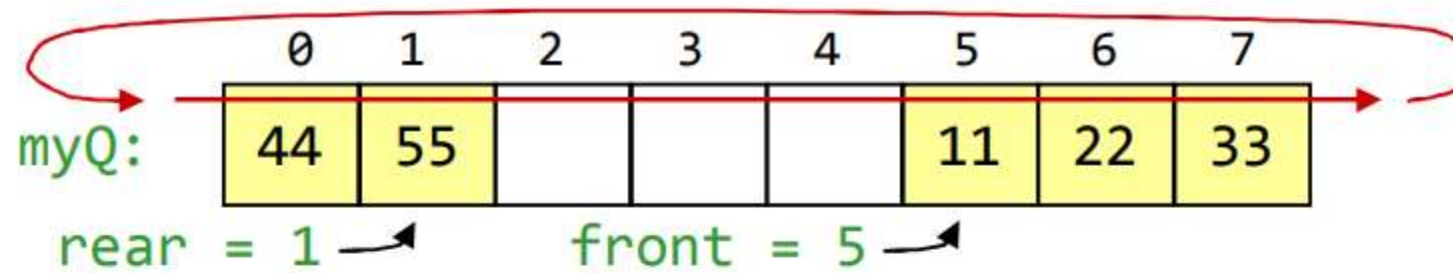
Circular Queues using Arrays

- We can treat the array holding the queue elements as circular (joined at the ends)



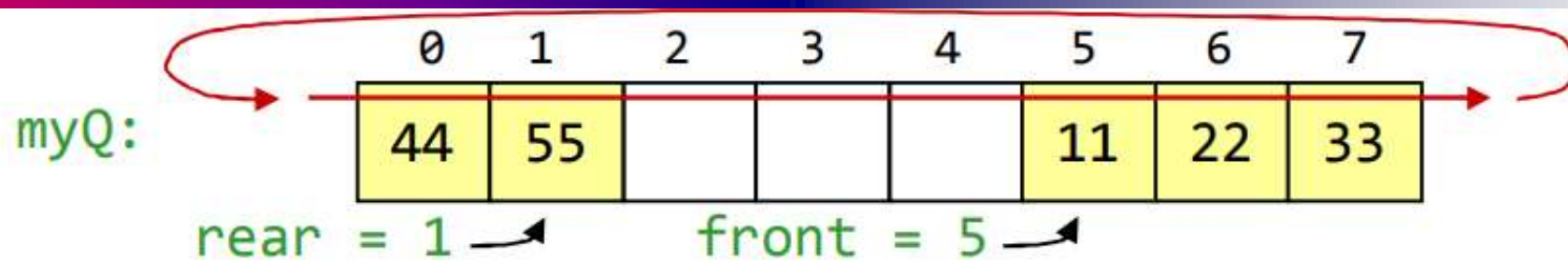
- The previous elements were deleted
- Elements were added to this queue in the order 11, 22, 33, 44, 55, and will be removed in the same order
- Use: $\text{front} = (\text{front} + 1) \% \text{Qsize}$; and: $\text{rear} = (\text{rear} + 1) \% \text{Qsize}$;

Circular Queues using Arrays: Enqueue()



```
void enqueue(int data) {  
    if(front == -1 && rear == -1) { // queue is empty  
        front = rear = 0;  
        myQ[rear]=data; }  
    else if((rear+1) % Qsize == front) // check queue is full  
        printf("Queue is overflow");  
    else {  
        rear=(rear+1) % Qsize; // rear is incremented  
        myQ[rear] = data; // assign a value  
    }  
}
```

Circular Queues using Arrays: Dequeue()

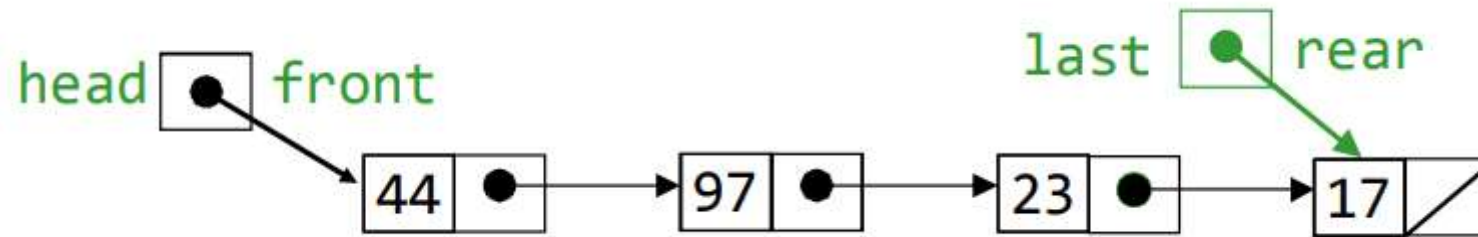


```
int dequeue() {  
    int y;  
    if((front == -1) && (rear == -1)) {  
        printf("\n Queue is underflow..");  
    }  
    else if(front == rear) { // there is only one element left  
        y = myQ[front]; front = -1; rear = -1; }  
    else {  
        y = myQ[front];  
        front = (front+1) % Qsize;  
    }  
    return y;  
}
```

Linked List Implementation of Queues

- In a queue, insertions occur at one end (rear end), deletions at the other end (front end).
- Operations at the head of a singly-linked list (SLL) are $O(1)$, but at the other end they are $O(n)$
 - Because you have to find the last element each time
- BUT: there is a simple way to use a singly-linked list to implement both insertions and deletions in $O(1)$ time
 - You always need a pointer to the first element in the list
 - You can keep an additional pointer to the last element in the list

Linked List Implementation of Queues



Enqueue (Insertion at the End)

```
struct Node {  
    int value;  
    struct Node* next;  
};  
struct Node* front, rear;
```

```
void enqueue(int data) {  
    struct Node* temp;  
    temp = (struct Node *)malloc(sizeof(struct Node));  
    // Check if memory(heap) is full.  
    if (!temp){  
        cout << "\n Heap Overflow";  
        exit(1);  
    }  
    temp->value = data;  
    rear->next = temp;  
    rear = temp;  
}
```

Deque (Deletion at the head and returning)

```
struct Node {  
    int value;  
    struct Node* next;  
};  
struct Node* front, rear;
```

```
int deQueue(){  
    struct Node* temp;  
    int data;  
    if (front == NULL) {  
        cout << "\n Queue underflow";  
        exit(1); }  
    else {  
        data = front->value  
        temp = front;  
        front = front->next;  
        free(temp);  
        return data;  
    }  
}
```

Queue Implementation Details

- With an array implementation:
 - you can have both overflow and underflow
- With a linked-list implementation: [?]
 - you can have underflow
 - overflow is a global out-of-memory condition



Thank You